Bitácora de desarrollo del Frontend

Feature 2: Análisis de impacto. AbastecePyme

Información general

Proyecto: AbastecePyme, interfaz de la red de dependencias de abastecimiento

Feature: Feature 2, Análisis de impacto (consulta y visualización)

Tecnología: React + TypeScript + Vite, con three.js (@react-three/fiber) para el grafo 3D

Asistente de IA: Claude Code (Anthropic)

Rama de trabajo: feature/impacto-api, integrada a main con el PR #4

Endpoint consumido: GET /impacto/{elemento_id}

Registro de prompts

Prompt 1: Consumo de la consulta de impacto desde la visualización

Herramienta: Claude Code

Prompt (versión redactada):

Conecta la vista de visualización con el endpoint GET /impacto/{id} del backend. Antes de escribir código, revisa cómo está organizado el proyecto (servicios en hooks/getProducts.ts, tipos en types.ts, componentes en features/graph) y respeta su estructura y estilo.

Contexto de negocio: cuando el coordinador de producción hace clic en un proveedor, insumo o producto, debe ver qué elementos de la red quedan afectados si ese elemento no está disponible.

El frontend no debe calcular el impacto. Hoy existe un cálculo local; reemplázalo para que toda la información venga de la respuesta del backend (origen, caso, mensaje, visitados y afectados con nivel y camino).

Requisitos:

1. Crear los tipos de la respuesta (ImpactoResponse, NodoAfectado) iguales al contrato del backend.
2. Crear la función getImpacto(id). Si el backend responde 404, convertirlo en el caso "inexistente" para que la vista maneje los tres casos de la misma forma.
3. Crear un hook que consulte el impacto cada vez que cambie el nodo seleccionado y que descarte la respuesta si el usuario cambia de nodo antes de que llegue.
4. Mostrar en el panel los tres casos (afecta_cadena, sin_dependencias, inexistente) con lenguaje de negocio, y para cada afectado indicar por qué camino le llega la falla.
5. Permitir apuntar al backend local con la variable VITE_API_URL.

Prompt 2: Animación de la falla por niveles en el grafo 3D

Herramienta: Claude Code (con el skill threejs-impl-react-three-fiber)

Prompt (versión redactada):

Usa el campo nivel que devuelve GET /impacto para animar el grafo 3D. El nodo seleccionado es el nivel 0. En lugar de pintar los afectados de rojo al instante, la falla debe avanzar como una

onda: primero el nivel 1, luego el nivel 2, con un pequeño retraso entre niveles. Las aristas que unen dos elementos afectados también se tiñen, y el resto de la red se apaga para que la cadena resalte.

Objetivo

Que el usuario de producción, con un solo clic, vea qué elementos de la red se caen cuando falla un proveedor o insumo, en qué orden y por qué camino, usando exclusivamente el resultado que calcula el backend.

Requerimientos de diseño solicitados

Sin cálculo de impacto en el frontend: el backend es la única fuente del recorrido.

Tipos de TypeScript alineados con el contrato ImpactoResponse del backend.

Los tres casos se muestran de forma diferenciada y comprensible.

Respuestas viejas descartadas cuando el usuario cambia de nodo.

Visualización por niveles, usando el nivel y el camino de cada afectado.

URL del backend configurable (Render por defecto, local con VITE_API_URL).

Resultado obtenido

Claude Code implementó la integración en el commit "consumo de GET /impacto y registro de proveedores e insumos", modificando estos archivos:

hooks/getProducts.ts: función getImpacto y manejo del 404 como caso inexistente.

hooks/useImpacto.ts: hook que consulta el impacto del nodo seleccionado.

types.ts: tipos CasoImpacto, NodoImpactado, NodoAfectado e ImpactoResponse.

components/VisualizationPage.tsx: panel de impacto con los tres casos y el camino de la falla.

components/Graph3D.tsx: recibe los niveles desde la API y anima la propagación.

Se eliminaron los hooks de cálculo local (useGraphImpact y useGraphImpactLevels).

Ajustes durante el desarrollo

Después del prompt principal se hicieron estos ajustes, a partir de la revisión del equipo:

Ajuste	Motivo
Quitar el cálculo local de impacto y usar solo la API.	La regla del curso es que los algoritmos de grafos viven en el backend; el front solo muestra el resultado.
Convertir el 404 en el caso "inexistente".	Que la vista trate los tres casos igual, sin mostrar un error genérico.
Descartar respuestas de un nodo anterior.	Si el usuario hace clic rápido en varios nodos, el panel no debe mezclar resultados.
Mostrar "vía ..." en los afectados de nivel 2 o más.	Que producción vea por qué elemento intermedio le llega la falla.
Animar la falla por niveles en lugar de pintar todo a la vez.	Que se entienda el efecto dominó: primero lo directo, luego lo indirecto.

Mejorar los estilos del panel de impacto.	Que el mensaje del backend y la lista de afectados se lean con claridad durante la demo.

Explicación del código

El frontend no recorre el grafo

El recorrido BFS y la decisión de cuál de los tres casos aplica están en el backend. El frontend solo hace la pregunta (GET /impacto/{id}) y dibuja la respuesta. Así no hay dos implementaciones del algoritmo que puedan dar resultados distintos.

getProducts.ts: la consulta

getImpacto llama al endpoint con el id del nodo seleccionado. Si la respuesta es 404, arma un objeto con caso "inexistente" y el mensaje del backend; si es otro error, lanza una excepción que el panel muestra.

```
export async function getImpacto(id: string): Promise<ImpactoResponse> {
  const response = await fetch(`${API_URL}/impacto/${encodeURIComponent(id)}`);
  if (response.status === 404) {
    return { origen: null, caso: "inexistente",
             mensaje: await getErrorDetail(response),
             visitados: [], afectados: [] };
  }
  if (!response.ok) throw new Error(`Error HTTP: ${response.status}`);
  return (await response.json()) as ImpactoResponse;
}
```

useImpacto.ts: el estado de la consulta

Cada vez que cambia el nodo seleccionado, el hook llama a getImpacto y entrega tres valores: impacto, error y cargando. Si el usuario cambia de nodo antes de que llegue la respuesta, la marca cancelado hace que esa respuesta se ignore.

VisualizationPage.tsx: el panel de impacto

Convierte la respuesta en un mapa de niveles (origen = 0) para el grafo y arma el mensaje del panel:

Caso	Qué ve el usuario
afecta_cadena	"Si este proveedor se detiene, quedan sin abastecer:" y la lista de afectados, con su capa y el camino ("vía Conservante alimentario").
sin_dependencias	"Nadie más en la red depende de esto. El fallo se queda aquí."
inexistente	El mensaje que devuelve el backend.

Graph3D.tsx: la animación por niveles

Cada afectado recibe su nivel. Una mancha se expande sobre el nodo con un retraso de 0,42 segundos por nivel, y las aristas entre dos afectados cambian de color al mismo ritmo. Los elementos no afectados se apagan.

Verificación

Prueba manual con el backend local: PRV-QUIM (afecta a conservante, ponqué y tostadas), PRV-MATERIAPRIMA (3 insumos en nivel 1 y 4 productos en nivel 2) y PRD-PAN (sin dependencias).

Revisión en la pestaña Network del navegador: cada clic genera una llamada a GET /impacto/{id}.

Comparación de la lista del panel con la respuesta en /docs del backend.

Pendiente: el archivo lib/impact.ts y su prueba son restos del cálculo local anterior; la interfaz ya no los usa y se eliminarán.